AI: Classical Planning

Planning Blocks World Problem

1 Introduction

Artificial Intelligence (AI) has been defined as the study of rational behavior, which implies that planning—formulating a course of action to accomplish specific goals—is a fundamental aspect of AI. In response to this, planning researchers have adopted a factored representation, where the state of the world is described by a set of variables. To express all actions using a single action schema, we use a language known as PDDL (Planning Domain Definition Language).

2 Example: The blocks world

The Block World Problem is a classic example used in artificial intelligence and classical planning. It involves a set of blocks, a table, and a robot capable of moving the blocks to achieve a specific configuration. The robot can perform actions like picking up, putting down, stacking, and unstacking blocks.

3 Problem Description

3.1 Initial State

In this new scenario, we have the following initial configuration:

- Block C is on top of block A, which is on the table.
- Block B is alone on the table.

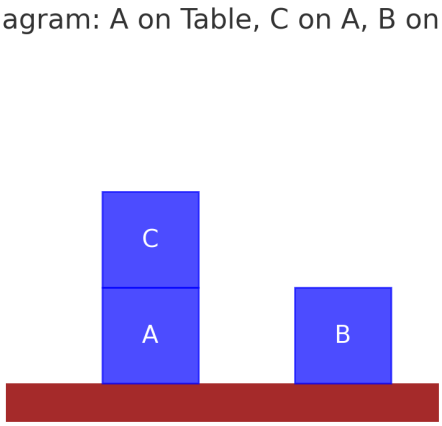


Figure 1: Initial State: A on C, B alone on the table.

3.2 Goal State

The goal is to rearrange the blocks as follows:

- Block C is on the table.
- Block B is on top of block C.
- Block A is on top of block B.

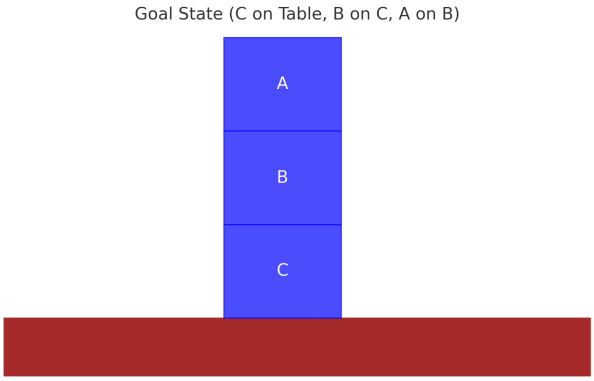


Figure 2: Goal State: A on B, B on C, C on the table.

4 Step-by-Step Solution

Here is the step-by-step plan for the robot to achieve the goal state from the initial state:

4.1 Step 1: Unstack C from A

Action: The robot picks up block C from block A.
Result: Block C is in the robot's hand, and block A is clear (nothing is on top of it).

Table: A B (C in hand)

4.2 Step 2: Put C on the table

Action: The robot places block C on the table.
Result: Block C is on the table, and the robot's hand is empty.

Table: A B C

4.3 Step 3: Pick up B from the table

Action: The robot picks up block B from the table.
Result: Block B is in the robot's hand, and block B is no longer on the table.

Table: A C (B in hand)

4.4 Step 4: Stack B on C

Action: The robot places block B on top of block C.
Result: Block B is on C, and the robot's hand is empty.

Table: A C
B (on C)

4.5 Step 5: Pick up A from the table

Action: The robot picks up block A from the table.
Result: Block A is in the robot's hand.

Table: C

B (on C) (A in hand)

4.6 Step 6: Stack A on B

Action: The robot places block A on top of block B.
Result: Block A is on B, and the robot's hand is empty.

Table: C

B (on C)

A (on B)

5 Conclusion

In this Block World Problem, starting from the initial state where block C is on block A and block B is alone on the table, the robot successfully rearranges the blocks into the goal state by performing six actions. The final arrangement has block A on block B, block B on block C, and block C on the table. This example demonstrates key classical planning techniques, such as search through action sequences and reasoning about preconditions and effects.

■

Block World Problem: PDDL Representation

Diagram: A on Table, C on A, B on Table

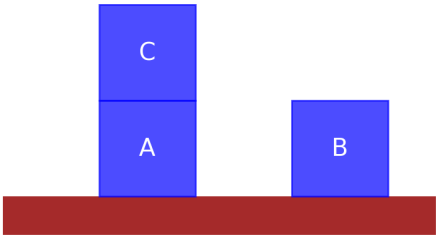


Figure 1: Initial State: A on C, B alone on the table.

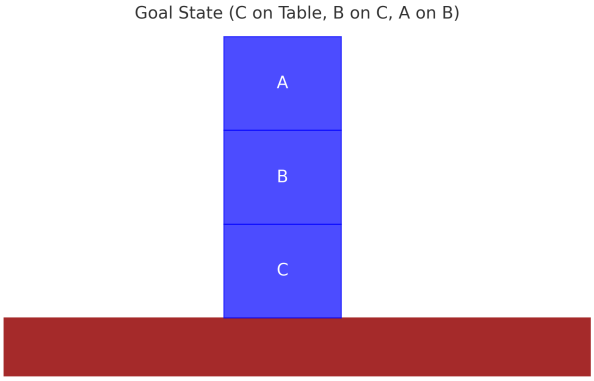


Figure 2: Goal State: A on B, B on C, C on the table.

Initial State:

$$\text{Init}(\text{On}(\text{A, Table}) \wedge \text{On}(\text{B, Table}) \wedge \text{On}(\text{C, A}) \wedge \text{Block}(\text{A}) \wedge \text{Block}(\text{B}) \wedge \text{Block}(\text{C}) \wedge \text{Clear}(\text{B}) \wedge \text{Clear}(\text{C}))$$

Goal State:

$$\text{Goal}(\text{On}(\text{A, B}) \wedge \text{On}(\text{B, C}))$$

Actions:

Move Action:

$$\text{Action(Move(b, x, y),}$$

PRECONDITION:

$$\text{On(b, x)} \ \wedge \ \text{Clear(b)} \ \wedge \ \text{Clear(y)} \ \wedge \ \text{Block(b)} \ \wedge \ \text{Block(y)} \wedge (b \neq x) \wedge (b \neq y) \wedge (x \neq y)$$

EFFECT:

$$\text{On(b, y)} \ \wedge \ \text{Clear(x)} \ \wedge \neg\text{On(b, x)} \ \wedge \neg\text{Clear(y)}$$

MoveToTable Action:

$$\text{Action(MoveToTable(b, x),}$$

PRECONDITION:

$$\text{On(b, x)} \ \wedge \ \text{Clear(b)} \ \wedge \ \text{Block(b)} \ \wedge (b \neq x)$$

EFFECT:

$$\text{On(b, Table)} \ \wedge \ \text{Clear(x)} \ \wedge \neg\text{On(b, x)}$$

Solution:

$$[\text{MoveToTable}(C, A), \text{Move}(B, \text{Table}, C), \text{Move}(A, \text{Table}, B)].$$

■

Air Cargo Transportation Problem

1 Problem Description

The Air Cargo Transportation Problem involves transporting cargo between multiple airports using airplanes. The objective is to move cargo from its initial locations to designated destinations, considering the availability of planes and the constraints on loading, unloading, and flying.

1.1 Example Setup

In this example, we have: - *Two airports*: $A1$ and $A2$. - *Two airplanes*: $P1$ and $P2$. - *Two cargo items*: $C1$ and $C2$.

1.2 Initial State

- *Cargo* $C1$ is at airport $A1$. - *Cargo* $C2$ is at airport $A2$. - *Airplane* $P1$ is at airport $A1$. - *Airplane* $P2$ is at airport $A2$.

$$\text{At(C1, A1)}, \quad \text{At(C2, A2)}, \quad \text{At(P1, A1)}, \quad \text{At(P2, A2)}$$

1.3 Goal State

The goal is to transport cargo such that: - *Cargo* $C1$ is at airport $A2$. - *Cargo* $C2$ is at airport $A1$.

$$\text{At(C1, A2)}, \quad \text{At(C2, A1)}$$

2 Actions

The following actions are available:

1. *Load(c, p, a)*: Load cargo c into airplane p at airport a.
- *Precondition*: Cargo c and airplane p are both at airport a.
- *Effect*: Cargo c is loaded onto airplane p and no longer at airport a.
2. *Unload(c, p, a)*: Unload cargo c from airplane p at airport a.
- *Precondition*: Cargo c is on airplane p and airplane p is at airport a.
- *Effect*: Cargo c is at airport a and no longer on airplane p.
3. *Fly(p, from, to)*: Fly airplane p from airport $from$ to airport to.
- *Precondition*: Airplane p is at airport $from$.
- *Effect*: Airplane p is now at airport to and no longer at airport $from$.

3 Solution: Sequence of Actions

To achieve the goal, we can apply the following sequence of actions:

1. *Load(C1, P1, A1)*: Load cargo $C1$ onto airplane $P1$ at airport $A1$.

$$\text{At(C2, A2)}, \quad \text{In(C1, P1)}, \quad \text{At(P1, A1)}, \quad \text{At(P2, A2)}$$

2. *Fly(P1, A1, A2)*: Fly airplane $P1$ from airport $A1$ to airport $A2$.

$$\text{At(C2, A2)}, \quad \text{In(C1, P1)}, \quad \text{At(P1, A2)}, \quad \text{At(P2, A2)}$$

3. *Unload(C1, P1, A2)*: Unload cargo $C1$ from airplane $P1$ at airport $A2$.

$$\text{At(C1, A2)}, \quad \text{At(C2, A2)}, \quad \text{At(P1, A2)}, \quad \text{At(P2, A2)}$$

4. *Load(C2, P2, A2)*: Load cargo $C2$ onto airplane $P2$ at airport $A2$.

$$\text{At(C1, A2)}, \quad \text{In(C2, P2)}, \quad \text{At(P1, A2)}, \quad \text{At(P2, A2)}$$

5. *Fly(P2, A2, A1)*: Fly airplane $P2$ from airport $A2$ to airport $A1$.

$$\text{At(C1, A2)}, \quad \text{In(C2, P2)}, \quad \text{At(P1, A2)}, \quad \text{At(P2, A1)}$$

6. *Unload(C2, P2, A1)*: Unload cargo $C2$ from airplane $P2$ at airport $A1$.

$$\text{At(C1, A2)}, \quad \text{At(C2, A1)}, \quad \text{At(P1, A2)}, \quad \text{At(P2, A1)}$$

After performing these actions, we reach the goal state where: - Cargo $C1$ is at airport $A2$. - Cargo $C2$ is at airport $A1$.

4 Conclusion

This example illustrates the use of a sequence of actions to achieve the desired configuration in the Air Cargo Transportation Problem. By applying the available actions (*load*, *unload*, and *fly*), we effectively move each piece of cargo to its destination. ■

Air Cargo Transportation Problem: PDDL Representation

Initial State

$$\begin{aligned}\text{Init}\big(&\text{At(C1, SFO)} \land \text{At(C2, JFK)} \land \text{At(P1, SFO)} \land \text{At(P2, JFK)} \\ &\land \text{Cargo(C1)} \land \text{Cargo(C2)} \land \text{Plane(P1)} \land \text{Plane(P2)} \\ &\land \text{Airport(JFK)} \land \text{Airport(SFO)}\big)\end{aligned}$$

Goal State

$$\text{Goal}\big(\text{At(C1, JFK)} \land \text{At(C2, SFO)}\big)$$

Actions

- **Action**(Load(c, p, a)):

  $$\text{PRECONDITION: At(c, a)} \land \text{At(p, a)} \land \text{Cargo(c)} \land \text{Plane(p)} \land \text{Airport(a)}$$

  $$\text{EFFECT: } \neg\text{At(c, a)} \land \text{In(c, p)}$$

- **Action**(Unload(c, p, a)):

  $$\text{PRECONDITION: In(c, p)} \land \text{At(p, a)} \land \text{Cargo(c)} \land \text{Plane(p)} \land \text{Airport(a)}$$

  $$\text{EFFECT: At(c, a)} \land \neg\text{In(c, p)}$$

- **Action**(Fly(p, from, to)):

  $$\text{PRECONDITION: At(p, from)} \land \text{Plane(p)} \land \text{Airport(from)} \land \text{Airport(to)}$$

  $$\text{EFFECT: } \neg\text{At(p, from)} \land \text{At(p, to)}$$

Action Sequence:

$$\begin{aligned}&[\text{Load}(C1, P1, \text{SFO}), \text{Fly}(P1, \text{SFO}, \text{JFK}), \text{Unload}(C1, P1, \text{JFK}), \\ &\quad \text{Load}(C2, P2, \text{JFK}), \text{Fly}(P2, \text{JFK}, \text{SFO}), \text{Unload}(C2, P2, \text{SFO})]\end{aligned}$$

■

Practice Example: The Spare Tire Problem

The Spare Tire Problem is a classic planning problem in Artificial Intelligence, designed to simulate real-world decision-making and action sequences. Imagine you are driving a car that has one spare tire, and suddenly, one of your tires gets punctured. The car has four mounted tires (T1, T2, T3, T4) and a single spare tire (TS). The goal is to replace the flat tire with the spare tire so the car can continue moving with four functional tires.

In order to solve this problem, you must pick up the spare, remove the flat tire, and install the spare tire in its place. This problem involves planning a sequence of actions, such as picking up, putting down, removing, and attaching tires, to achieve the goal. Planning languages like **PDDL** (Planning Domain Definition Language) are used to formally describe the actions, preconditions, and effects needed to solve such problems.

Question: Based on the problem definition, write a PDDL representation for the Spare Tire Problem. Your PDDL should include:

1. **Initial State:** Specify the initial conditions, such as which tire is flat, the location of each tire, and that the car has a spare tire available.
2. **Goal State:** Define the desired end state where all four positions on the car have a functioning tire, and the flat tire is replaced by the spare.
3. **Actions:** List the actions available to achieve the goal, with each action including:
    - **Preconditions:** Conditions that must be true before the action can occur.
    - **Effects:** Changes to the state after the action is completed.

Consider actions such as *remove, attach, pick up,* and *put down* in your representation, ensuring that each action includes relevant preconditions and effects.